অপারেটর এবং পর্দায় ছাপানো

পাঠ 4 — 1IOT, SST-ICT-01 — গাণিতিক অপারেটর, মডুলো, মান নির্ধারণ, printf

21/09/2026

1 বোর্ডে পুনরালোচনা: তিনটি ডেটা টাইপ

অপারেটর শুরু করার আগে, গত পাঠে যা দেখেছি তার একটি ছোট চিত্র, যেমনটি **বোর্ডে** (**lavagna**) লেখা হয়েছিল:

8 BIT → `char` ┐
16 BIT → `int` ┘ পূর্ণসংখ্যা
32 BIT → `float` ┘ বাস্তব সংখ্যা

ছবি/নকশা 1: বোর্ডে যেমন লেখা হয়েছিল: char আর int একসাথে "পূর্ণসংখ্যা", আর float আলাদা, "বাস্তব সংখ্যার" (দশমিকসহ সংখ্যা) টাইপ হিসেবে।

2 অপারেটর কী

সংজ্ঞা

অপারেটর (**operatore**) হলো সেই **প্রতীক**, যা দুই বা তার বেশি **অপারেন্ডের** (**operandi**, উপাদান) মাঝে বসে তাদের **একত্র করে** একটি **ফলাফল** দেয়। কোন প্রতীক ব্যবহার করা হচ্ছে তার ওপর **আলাদা কাজ** নির্ভর করে।

উদাহরণ

গণিতে $5 + 2 = 7$: আমি 5 আর 2 নিয়ে যোগের **অপারেটর** দিয়ে একত্র করি, পাই 7। **গুণ** দিয়ে একত্র করলে পাই 10: একই অপারেন্ড, ভিন্ন অপারেটর, **ভিন্ন ফলাফল**।

3 গাণিতিক অপারেটর

সংজ্ঞা

গাণিতিক অপারেটর হলো প্রাথমিক স্কুলে শেখা অপারেটরগুলো: `+` (**যোগ**, **somma**), `-` (**বিয়োগ**, **sottrazione**), `*` (**গুণ**, **moltiplicazione**, তারকাচিহ্ন), `/` (**ভাগ**, **divisione**, একে বলে **slash**)।

টীকা

কম্পিউটারে **ভাগ** লেখা হয় **slash** (`/`) দিয়ে, গণিতে ব্যবহৃত **কোলন** বা নামতার ভাগচিহ্ন দিয়ে নয়।

3.1 মডুলো অপারেটর

সংজ্ঞা

% প্রতীকটি (দেখতে **শতাংশের** মতো, কিন্তু **তা নয়**) হলো **মডুলো অপারেটর** (**operatore modulo**): এটি **পূর্ণসংখ্যার ভাগের ভাগশেষ** ফেরত দেয়।

উদাহরণ

পূর্ণসংখ্যায় $5 \div 2$ কত? ভাগের ফলাফল 2, **ভাগশেষ 1**। তাই $5 \bmod 2 = 1$।
$11 \bmod 3$ কত? $11 \div 3 = 3$ (কারণ $3 \times 3 = 9$), আর $11 - 9 = 2$: তাই $11 \bmod 3 = 2$।
$100 \bmod 30$ কত? $100 \div 30 = 3$ (কারণ $3 \times 30 = 90$), আর $100 - 90 = 10$: তাই $100 \bmod 30 = 10$।
বোর্ডে করা আরও উদাহরণ: $16 \bmod 5 = 1$, $40 \bmod 9 = 4$।

সতর্কতা

মডুলো **শুধু ভাগশেষ** দেয়, **ভাগের ফলাফল** নয়। ভাগের মানের জন্য **slash** (/) লাগে। এই অপারেটরটি ভালোভাবে বোঝা **খুব জরুরি**: সামনে প্রোগ্রামিংয়ে এটি অনেক ব্যবহার হবে।

4 মান নির্ধারণের অপারেটর

সংজ্ঞা

প্রোগ্রামিংয়ে = প্রতীকটি হলো **মান নির্ধারণের অপারেটর** (**operatore di assegnazione**): এটি তার **ডান দিকের** মান **কপি করে** তার **বাঁ দিকের ভেরিয়েবলে** রাখে।



ছবি/নকশা 2: মান নির্ধারণের অপারেটরের বোর্ডের চিত্র: ডান বাক্সের মান বাঁ বাক্সে কপি হয় – তীর সবসময় ডান থেকে বাঁয়ে যায়।

উদাহরণ

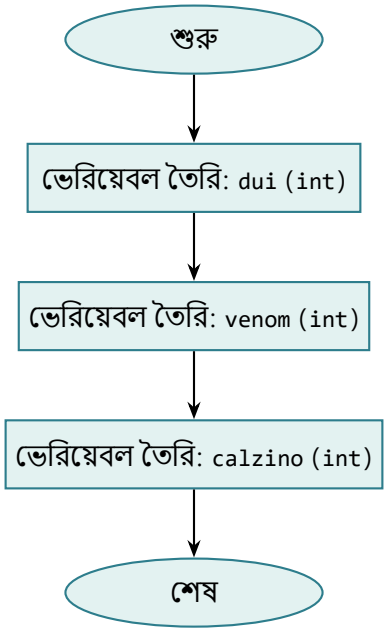
`int` টাইপের তিনটি **ভেরিয়েবল** তৈরি করি, **নিজের ইচ্ছামতো নাম** দিয়ে (যেমন `dui`, `venom`, `calzino` – নামের সাথে তার অর্থের সম্পর্ক থাকতেই হবে এমন নয়, তবে একটি **অর্থবহ নাম** পরে কোড পড়তে সাহায্য করে):

```
int dui;
int venom;
int calzino;
```

প্রতিটি লাইনের শেষে **সেমিকোলন** (**punto e virgola**) ভুলো না: এটি নির্দেশটি শেষ করে। তারপর মান দিই:

```
dui = 7;
venom = 10;
calzino = dui - venom;
```

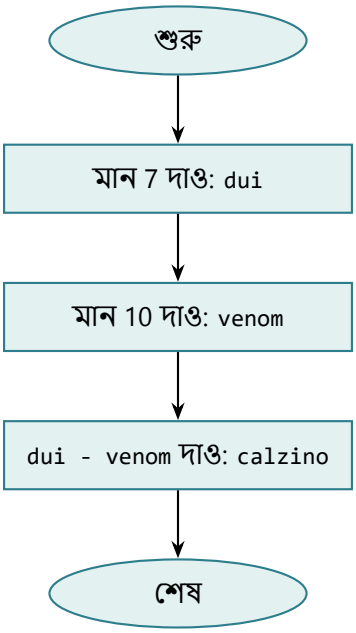
এখানে `dui`-তে 7, `venom`-এ 10, আর `calzino` পায় `dui - venom`-এর **ফলাফল**, অর্থাৎ -3: কম্পিউটার **কাজটি করে** এবং যে মান বের হয় তা বাঁয়ের ভেরিয়েবলে কপি করে।



ছবি/নকশা 3: ফ্লোচার্ট (**diagramma di flusso**): তিনটি `int` ভেরিয়েবলের ধারাবাহিক ঘোষণা।

টীকা

অপারেটর আর সংখ্যার মাঝের **ফাঁক** খেয়াল করো: `dui = 7;` ঠিক `dui=7;` (ফাঁক ছাড়া)-এর মতোই কাজ করে – কম্পাইলারের কাছে **কোনো পার্থক্য নেই** – তবে ফাঁকসহ লেখা পড়তে সহজ, তাই সেটাই ব্যবহার করা ভালো।



ছবি/নকশা 4: ফ্লোচার্ট: তিনটি মান নির্ধারণ একের পর এক, কোডে লেখা ক্রমে।

সতর্কতা

প্রোগ্রামিংয়ে = প্রতীকটি গণিতের **সমান চিহ্ন নয়**। গণিতে "$x = 3$" x আর 3-এর সম্পর্ক বলে, x-এর কোন মানে কথাটি সত্য তা খোঁজে। প্রোগ্রামিংয়ে = **সবসময় ডান থেকে বাঁয়ে কপি করে**: `5 = a` লেখার কোনো মানে নেই, কারণ বাঁয়ে কোনো **ভেরিয়েবল** নেই (5-এর মতো সংখ্যা অন্য মান "**ধারণ**" করতে পারে না) – এতে প্রায় সবসময় **কম্পাইলের ভুল** (**errore di compilazione**) হয়।

5 ভেরিয়েবলের নামের নিয়ম

সংজ্ঞা

একটি **ভেরিয়েবলের** নামকে এই নিয়মগুলো মানতে হবে:

1. এটি একটি **অক্ষর-সংখ্যার** ধারা: এতে **অক্ষর** ও **সংখ্যা** থাকতে পারে।
2. এটি **সংখ্যা দিয়ে শুরু** হতে পারে না (সংখ্যা ব্যবহার করা যায়, তবে প্রথম অক্ষর হিসেবে নয়)।
3. এতে **বিশেষ অক্ষর থাকতে পারে না**, শুধু **underscore** (_, **trattino basso**) ছাড়া – এটিই একমাত্র অনুমোদিত ব্যতিক্রম।
4. এটি **case sensitive**: `pippo` আর `Pippo` দুটি **আলাদা** ভেরিয়েবল।
5. **এটি ভাষার কোনো সংরক্ষিত শব্দ হতে পারে না** (যেমন `int`, `char`, `return`, `for`): এই শব্দগুলো C ভাষা নিজের জন্য ব্যবহার করে, ভেরিয়েবলের নাম হিসেবে আবার ব্যবহার করা যায় না।

উদাহরণ

বোর্ডে করা উদাহরণ, যেখানে ক্লাসকে ঠিক করতে হয়েছিল প্রতিটি নাম বৈধ কিনা:

- `MICHELE` – **বৈধ**।
- `pippo11` – **বৈধ** (অক্ষর+সংখ্যা, **সংখ্যা দিয়ে শুরু** নয়)।
- `12LUIGI` – **বৈধ নয়**: **সংখ্যা দিয়ে শুরু**।
- `MAURO_BORTOLANI` – **বৈধ**: **underscore** একমাত্র অনুমোদিত বিশেষ অক্ষর।
- `MAURO BORTOLANI` (একটি **ফাঁক** সহ) – **বৈধ নয়**: ফাঁক ধারাটি ভেঙে দেয়, ভাষার কাছে নামটি দুটি আলাদা অংশ হয়ে যায়।
- `a,b-5` – বৈধ **নয়**: **কমা** আর ড্যাশ অননুমোদিত **বিশেষ অক্ষর**।

উদাহরণ

বাস্তবে case sensitive: `pippo`, `Pippo` আর `PippoO` **কম্পাইলারের** চোখে **তিনটি আলাদা ভেরিয়েবল**, যদিও একজন মানুষের কাছে "**প্রায় একই**" মনে হয়।

সতর্কতা

নাম **underscore** দিয়ে শুরু করা যায়, কিন্তু C ভাষায় এভাবে শুরু হওয়া ভেরিয়েবলের একটি বিশেষ **কারিগরি অর্থ** আছে (সিস্টেম/নির্দেশনার ভেরিয়েবল), যা আমরা এই কোর্সে ব্যবহার করব না – বিভ্রান্তি এড়াতে এড়িয়ে চলাই ভালো।

উদাহরণ

ক্লাসে নির্দেশিত অনুশীলন – তিনটি **ভেরিয়েবল** ঘোষণা করি এবং মডুলো দিয়ে একটি হিসাবে ব্যবহার করে দেখি:

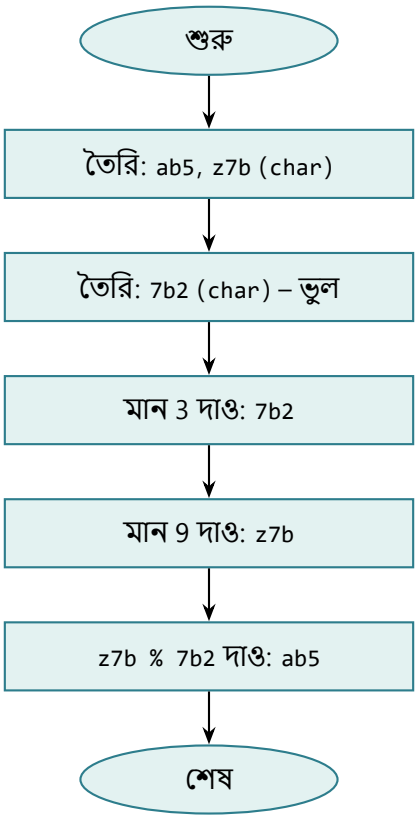
```
char ab5;
char z7b;
char 7b2;       // ভুল: নাম সংখ্যা দিয়ে শুরু হয়েছে!

7b2 = 3;
z7b = 9;
ab5 = z7b % 7b2;
```

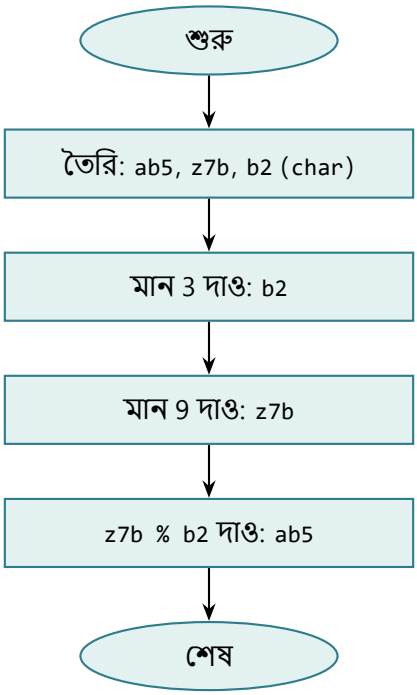
7b2 নামটি **নিয়ম** **2** ভাঙে (সংখ্যা দিয়ে শুরু হতে পারে না): প্রোগ্রাম **কম্পাইল** **হয়** **না**। **শুরুর** **সংখ্যাটি** সরিয়ে ঠিক করা হলো:

```
char ab5;
char z7b;
char b2;        // ঠিক: এখন অক্ষর দিয়ে শুরু

b2 = 3;
z7b = 9;
ab5 = z7b % b2;    // ab5 পায় 9 % 3 = 0
```



ছবি/নকশা 5: ভুল সংস্করণের ফ্লোচার্ট: 7b2 নামটি অবৈধ হওয়ায় কম্পাইলের সময় ধারাটি থেমে যায়।



ছবি/নকশা 6: সঠিক সংস্করণের ফ্লোচার্ট: শেষে ab5 পায় $9 \bmod 3 = 0$ মান।

6 ফলাফল পর্দায় ছাপানো: printf

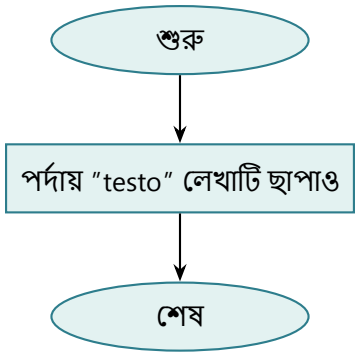
এখন পর্যন্ত যন্ত্র হিসাব করে, কিন্তু আমাদের দেখায় না: **printf** নির্দেশ দিয়ে **মনিটরে** আমাদের কাজের **ফলাফল** **দেখানোর** একটি উপায় দরকার।

সংজ্ঞা

`printf` নির্দেশটি (সব ছোট হাতের অক্ষরে) মনিটরে একটি লেখা **ছাপায়** (**stampa**)। এভাবে লেখা হয়:

```
printf("testo");
```

লেখাটি গোল বন্ধনীর ভেতরে, **ডাবল** **কোটেশনের** (", **doppi apici**) মধ্যে রাখতে হয়।



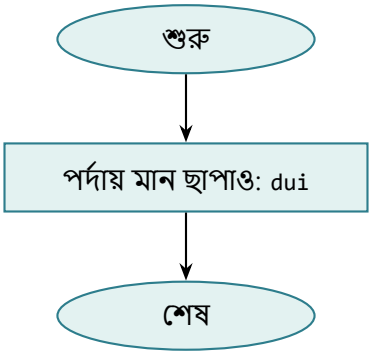
ছবি/নকশা 7: ফ্লোচার্ট: একটিমাত্র কাজ, মনিটরে লেখা ছাপানো।

উদাহরণ

লেখার ভেতরে একটি **ভেরিয়েবলের মান** ছাপাতে একটি **ফরম্যাট নির্দেশক** (**specificatore di formato**) ব্যবহার হয়: `%d` প্রতীকটি (পার্সেন্ট-ডি), যেখানে **মানটি দেখাতে** চাই সেখানে বসানো হয়।

```
printf("%d", dui);
```

`dui` ভেরিয়েবলের মান -3 হলে এই নির্দেশ **পর্দায় ছাপায়**: `-3`।



ছবি/নকশা 8: ফ্লোচার্ট: একটিমাত্র কাজ, `%d` দিয়ে `dui` ভেরিয়েবলের মান ছাপানো।

টীকা

এখানে `%` প্রতীকের সাথে **গণিতের শতাংশের** কোনো সম্পর্ক নেই: এটি একটি **ফরম্যাট নির্দেশক** শুরু করে, অর্থাৎ একটি **বিশেষ নির্দেশ** যা বলে "এই জায়গায় একটি ভেরিয়েবলের মান বসাও"; এটি ছাপানোর মতো কোনো **আক্ষরিক অক্ষর** নয়।
একে **escape ধারার** (**sequenze di escape**) সাথে গুলিয়ে ফেলো না (যেমন **নতুন লাইনে যেতে** `\n`, ট্যাবের জন্য `\t`): সেগুলো backslash `\` দিয়ে শুরু হয় এবং লেখায় একটি **বিশেষ অক্ষর** বোঝায়, আর `%d` `printf`-কে বলে একটি মান কীভাবে **বুঝতে ও দেখাতে** হবে – এগুলো **দুটি আলাদা ব্যবস্থা**, যদিও একই ডাবল কোটেশনের ভেতরে একসাথে ব্যবহার হয়।

সতর্কতা

`printf`-এর ভেতরের **ফাঁক** আর বিরামচিহ্ন খেয়াল করো: একটি **ভুলে যাওয়া কমা** বা ভুল ফাঁকে কম্পাইল ব্যর্থ হয়। কিছু ইনস্টল না করে এই প্রথম পরীক্ষাগুলো করার একটি ব্যবহারিক পরিবেশ হলো onlinegdb.com: এটি পেশাদার সরঞ্জাম নয়, কিন্তু শুরু করার জন্য সুবিধাজনক, যেকোনো ব্রাউজারে চলে (ট্যাবলেট বা স্মার্টফোনেও)।

7 onlinegdb.com-এ ব্যবহারিক অনুশীলন

এই পাঠে আমরা একসাথে একটি ছোট **প্রোগ্রাম** লিখে চালিয়েছি, **অনলাইন পরিবেশে** আগে থেকে থাকা **উদাহরণ কোড** থেকে শুরু করে। আমাদের দরকারি অংশ সবসময় `main` ফাংশনের দুই **কার্লি বন্ধনীর** (**parentesi graffe**) মাঝের অংশ (অর্থাৎ খোলা ও বন্ধ বন্ধনীর মাঝের সবকিছু): সেখানে উদাহরণের লেখা মুছে **নিজের কোড লিখি**।

টীকা

ইতালীয় কিবোর্ডে কার্লি বন্ধনী লেখার উপায়: খোলা বন্ধনী `{` পাওয়া যায় `Alt Gr + Shift + è` দিয়ে (দুই বর্গবন্ধনীর কী); বন্ধ বন্ধনী `}` পাওয়া যায় `Alt Gr + Shift +` পাশের কী দিয়ে। প্রোগ্রামিংয়ে এগুলো সবচেয়ে বেশি ব্যবহৃত প্রতীকগুলোর মধ্যে: এই সংমিশ্রণ **এখনই মুখস্থ করা** ভালো।

সতর্কতা

সাধারণ ভুল: প্রোগ্রামের শেষে **বন্ধ বন্ধনী ভুলে গেলে কম্পাইলের ভুল** হয়। প্রোগ্রাম চালু না হলে এবং কারণ স্পষ্ট না হলে, প্রথমে দেখো প্রতিটি **খোলা বন্ধনীর** একটি মিলে যাওয়া **বন্ধ বন্ধনী** আছে কিনা।

সতর্কতা

একটি আরও চালাক সমস্যা: ব্রাউজারে পাতার **স্বয়ংক্রিয় অনুবাদ** চালু থাকলে (যেমন Google-এর অন্তর্নির্মিত অনুবাদক), প্রোগ্রামিং পরিবেশে ভুলভাবে অনূদিত লেখা দেখা যেতে পারে – যেমন **মূল শব্দ** `return` অনূদিত হয়ে দেখা দিতে পারে এবং C ভাষার শব্দ হিসেবে **আর চেনা না যেতে পারে**, যা অদ্ভুত ও বোঝা-কঠিন ভুল তৈরি করে। এমন হলে ব্রাউজারের সেটিংস থেকে পাতার **স্বয়ংক্রিয় অনুবাদ বন্ধ করতে** হবে।

কোড লেখার পর, **Run** বোতাম (ওপরে, সবুজ রঙের) **প্রোগ্রাম চালায়**। প্রত্যাশিত ফলাফলের বদলে ভুল দেখালে, কারণ প্রায় সবসময় এগুলোর একটি: একটি **ভুলে যাওয়া বন্ধনী**, একটি **কমা** বা **সেমিকোলন না থাকা**, অথবা ঘোষণার চেয়ে আলাদাভাবে লেখা একটি **ভেরিয়েবলের নাম** (মনে রেখো: C ভাষা **case sensitive**)।

৪ এই পাঠ থেকে যা মনে রাখতে হবে

- **গাণিতিক অপারেটর**: `+ - * /` (শেষেরটির নাম **slash**)।
- **মডুলো অপারেটর** (`%`) পূর্ণসংখ্যার ভাগের **ভাগশেষ** দেয়, **ভাগের ফলাফল** নয়।
- **মান নির্ধারণের অপারেটর** (`=`) **ডানের** মান **বাঁয়ের** ভেরিয়েবলে কপি করে – এটি **গণিতের সমান নয়**।
- ভেরিয়েবলের নামের নিয়ম: **অক্ষর-সংখ্যা**, **সংখ্যা দিয়ে শুরু** নয়, **বিশেষ অক্ষর** নয় (**underscore** ছাড়া), **case sensitive**।
- `printf("%d", variabile);` **ফরম্যাট নির্দেশক** `%d` দিয়ে একটি ভেরিয়েবলের মান **পর্দায় ছাপায়**।
- ইতালীয় কিবোর্ডে **কার্লি বন্ধনী** লেখা হয় `Alt Gr + Shift +` সংশ্লিষ্ট কী দিয়ে – আর এগুলো সবসময় **জোড়ায়** থাকে (খোলা/বন্ধ)।

উদাহরণ

পরের পাঠের জন্য করণীয়: দুটি `char` ভেরিয়েবল তৈরি করো, ইচ্ছামতো দুটি মান দাও, তাদের **যোগফল** আর ভাগের **ভাগশেষ** (মডুলো অপারেটর) হিসাব করো, এবং `printf` দিয়ে দুটি ফলাফলই **ছাপাও**।

9 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **operatore / operando**: অপারেটর / অপারেন্ড
- **lavagna**: বোর্ড
- **resto**: ভাগশেষ
- **assegnazione**: মান নির্ধারণ
- **punto e virgola**: সেমিকোলন (;)
- **parentesi graffe / tonde**: কার্লি বন্ধনী { } / গোল বন্ধনী ( )
- **stampare a video**: পর্দায় ছাপানো
- **diagramma di flusso**: ফ্লোচার্ট
- **errore di compilazione**: কম্পাইলের ভুল
- **dichiarare**: ঘোষণা করা
- **doppi apici**: ডাবল কোটেশন (")